Lab 4: Vibe Coding - LLM Chat History

Student USN: PES1UG24CS444
Repository: Scenario 16 - Hangman Challenge (16_hangman)
Tool Used: Antigravity CLI (agy) / Gemini 3.8 Flash

=== User ===

I am working on a terminal Hangman game in Python with existing files: main.py, game.py, words.py, and stats.py.
All state must remain strictly in memory (no external files/JSON/DB).
Do not rewrite the entire structure; modify the existing code cleanly.

Here is the current game.py:
@[game.py]

Help with Task 1:
In

```
guess()
```

, repeated wrong guesses currently consume extra lives because the code only checks

```
if letter in self.guessed
```

.
Fix the guess handling so that:
1. If a letter has already been guessed (either in

```
self.guessed
```

or

```
self.wrong
```

), return "Already guessed." without deducting lives or
altering game state.
2. Each distinct letter affects the round exactly once.
Provide the updated game.py.

<ADDITIONAL_METADATA>
The current local time is: 2026-10-11T16:32:49+05:30.

The user has mentioned some items in the form @[ITEM]. Here is extra information about the items that were mentioned by the user, in the order that they appear:

@[game.py] is a [File]:
C:\Abhyaas\College\sem5\03_SE\Lab\lab_4\16_hangman\game.py
</ADDITIONAL_METADATA>
<USER_SETTINGS_CHANGE>
The user changed setting

```
Model Selection
```

from None to Gemini 3.8 Flash (Medium). No need to comment on this change if the user doesn't ask about it. If reporting what model you are, please use a human readable name instead of the exact string.
</USER_SETTINGS_CHANGE>

=== Antigravity (AI Assistant) ===

The guess handling in [

```
HangmanGame.guess()
```

](file:///C:/Abhyaas/College/sem5/03_SE/Lab/lab_4/16_hangman/game.py#L29-L40) has been fixed.

Summary of Changes:
1. Checked both

```
self.guessed
```

and

```
self.wrong
```

before checking the secret word:

```
   if letter in self.guessed or letter in self.wrong:
       return "Already guessed."
```

2. Added

```
letter = letter.lower()
```

defensively inside

```
guess()
```

to ensure consistent letter comparison regardless of caller input casing.
3. If a letter has already been attempted (correct or wrong), it returns

```
"Already guessed."
```

without modifying lives or adding duplicates to

```
self.wrong
```

/

```
self.guessed
```

.

Updated [game.py](file:///C:/Abhyaas/College/sem5/03_SE/Lab/lab_4/16_hangman/game.py)

```
import random
from words import WORDS, HINTS


class HangmanGame:
    def __init__(self):
        self.score = 0
        self.streak = 0
        self.category = "technology"
        self.secret = ""
        self.guessed = set()
        self.wrong = set()
        self.lives = 6
        self.hint_used = False

    def start_round(self):
        self.secret = random.choice(WORDS[self.category])
        self.guessed.clear()
        self.wrong.clear()
        self.lives = 6
        self.hint_used = False

    def masked(self):
        return " ".join(ch if ch in self.guessed else "_" for ch in self.secret)

    def won(self):
        return all(ch in self.guessed for ch in set(self.secret))

    def guess(self, letter):
        if len(letter) != 1 or not letter.isalpha():
            return "Enter one letter."
        letter = letter.lower()
        if letter in self.guessed or letter in self.wrong:
            return "Already guessed."
        if letter in self.secret:
            self.guessed.add(letter)
```

```
            return "Correct."
        self.wrong.add(letter)
        self.lives -= 1
        return "Wrong."

    def use_hint(self):
        if self.hint_used:
            return None
        self.hint_used = True
        self.score = max(0, self.score - 1)
        return HINTS.get(self.secret, "No hint available.")

    def play_round(self):
        self.start_round()
        while self.lives &gt; 0 and not self.won():
            print("\nWord:", self.masked())
            print("Wrong:", " ".join(sorted(self.wrong)) or "-")
            print("Lives:", self.lives, "Score:", self.score, "Streak:", self.streak)
            raw = input("Letter, /hint, or /quit: ").strip().lower()
            if raw == "/quit":
                return False
            if raw == "/hint":
                hint = self.use_hint()
                print(hint if hint else "Hint already used.")
                continue
            print(self.guess(raw))

        if self.won():
            self.streak += 1
            self.score += 5 + self.streak
            print("Solved:", self.secret)
            return True

        self.streak = 0
        print("Out of lives. The word was:", self.secret)
        return True

    def run(self):
        print("Hangman Challenge")
        print("A session consists of multiple rounds.")
        while True:
            print("\nCategories:", ", ".join(WORDS))
            raw = input("Choose category or q: ").strip().lower()
            if raw == "q":
                return
            if raw not in WORDS:
                print("Unknown category.")
                continue
            self.category = raw
            if not self.play_round():
                return
            again = input("Another round? [y/n]: ").strip().lower()
            if again != "y":
                print("Final score:", self.score, " Streak:", self.streak)
                return
```

=== User ===

Great! Task 1 is working. Now in this turn, let's complete Task 2, Task 3, and Task 4 together:

Context on existing stats.py:
from stats import SessionStats
class SessionStats:
def __init__(self):
self.rounds = 0
self.wins = 0
self.best_streak = 0
def record(self, won, streak):
self.rounds += 1
self.wins += int(won)
self.best_streak = max(self.best_streak, streak)

Requirements:
1. Task 2 - Complete the session model:
- Import and integrate

```
SessionStats
```

from

```
stats.py
```

into

```
HangmanGame
```

.
- Update stats after each round using

```
self.stats.record(won, self.streak)
```

.
- Ensure starting a new round resets only round-specific state (secret, guessed, wrong, lives, hint_used), while session stats (rounds,
wins, best streak, total score) persist across the entire run.
- When the user quits or ends the session, print full summary statistics (Rounds played, Wins, Win percentage, Best streak, Total score).

2. Task 3 - Difficulty and scoring:
- Add a difficulty selection (e.g. Easy: 8 lives, Medium: 6 lives, Hard: 4 lives, or configurable multipliers for score).
- Prompt the user to select difficulty (or retain selected difficulty) without leaking round state.
- Hint usage should consistently deduct points (e.g. -2 points, or scaled penalty) without letting score drop below 0.

3. Task 4 - Robust input and feedback:
- Handle invalid letters, non-alphabet characters, and malformed slash commands gracefully without changing game state or deducting
lives.
- For replay prompts ("Another round? [y/n]"), accept case-insensitive inputs like 'y', 'yes', 'n', 'no'.
- Ensure feedback messages (e.g., "Already guessed.", "Wrong.", "Correct.") are displayed cleanly without duplicate output.
- Keep all state in memory (no external file/DB storage).

Please provide the complete, updated code for

```
game.py
```

(and any updates to

```
stats.py
```

or

```
main.py
```

if necessary).

<ADDITIONAL_METADATA>
The current local time is: 2026-10-11T16:38:41+05:30.
</ADDITIONAL_METADATA>

=== Antigravity (AI Assistant) ===

Here is the completed implementation for **Tasks 2, 3, and 4**.

Key Implementations

1. **Task 2 - Session Model (

`stats.py`

&

`game.py`

)**:
- Integrated [

`SessionStats`

](file:///C:/Abhyaas/College/sem5/03_SE/Lab/lab_4/16_hangman/stats.py) into [

`HangmanGame`

](file:///C:/Abhyaas/College/sem5/03_SE/Lab/lab_4/16_hangman/game.py).
- Added a

`win_percentage`

property in

`SessionStats`

(handling the division-by-zero case when rounds played is

`0`

).
- Recorded results after each round using

`self.stats.record(won, self.streak)`

.
- Ensured

`start_round()`

resets only round-specific state (

`secret`

,

`guessed`

,

`wrong`

,

`lives`

,

`hint_used`

), while

`stats`

,

`score`

, and

`streak`

persist across rounds.
- Added

`print_summary()`

to output full session statistics (*Rounds played*, *Wins*, *Win percentage*, *Best streak*, and *Total score*) whenever the session ends.

2. **Task 3 - Difficulty & Scoring**:

- Added

```
DIFFICULTIES
```

configuration:
- **Easy**: 8 lives, 5 base points.
- **Medium**: 6 lives, 10 base points.
- **Hard**: 4 lives, 15 base points.
- User is prompted to select or retain their chosen difficulty (pressing <kbd>Enter</kbd> keeps the current difficulty). Round rules adapt dynamically without state leaks.
- Hint usage consistently deducts 2 points without dropping the score below 0 (

```
max(0, self.score - 2)
```

).

3. **Task 4 - Robust Input & Feedback**:
- Handled empty entries, non-alphabet inputs, multi-character entries, and malformed slash commands (e.g.,

```
/unknown
```

) cleanly without mutating game state or consuming lives.
- Replay prompt accepts case-insensitive responses (

```
'y'
```

,

```
'yes'
```

,

```
'n'
```

,

```
'no'
```

) and reprompts on invalid entries.
- Feedback messages (

```
"Already guessed."
```

,

```
"Wrong."
```

,

```
"Correct."
```

) are displayed cleanly without duplicate output.
- All state remains strictly in-memory.

Updated Code

1. [stats.py](file:///C:/Abhyaas/College/sem5/03_SE/Lab/lab_4/16_hangman/stats.py)

```
class SessionStats:
    def __init
&lt;truncated 3691 bytes&gt;
mes}) or press Enter to keep [{self.difficulty}]: ").strip().lower()
            if not raw:
                return True
            if raw in ("q", "quit", "/quit"):
                return False
            if raw in DIFFICULTIES:
                self.difficulty = raw
                return True
            print("Unknown difficulty. Please choose from:", ", ".join(DIFFICULTIES.keys()))
    def choose_category(self):
```

```
    while True:
        print("\nCategories:", ", ".join(WORDS))
        raw = input("Choose category or q: ").strip().lower()
        if raw in ("q", "quit", "/quit"):
            return None
        if raw in WORDS:
            return raw
        print("Unknown category. Please choose from:", ", ".join(WORDS))

def ask_replay(self):
    while True:
        again = input("\nAnother round? [y/n]: ").strip().lower()
        if again in ("y", "yes"):
            return True
        if again in ("n", "no"):
            return False
        print("Invalid input. Please enter 'y' or 'n'.")

def print_summary(self):
    print("\n=== Session Statistics ===")
    print(f"Rounds played : {self.stats.rounds}")
    print(f"Wins          : {self.stats.wins}")
    print(f"Win percentage: {self.stats.win_percentage:.1f}%")
    print(f"Best streak   : {self.stats.best_streak}")
    print(f"Total score   : {self.score}")

def run(self):
    print("Hangman Challenge")
    print("A session consists of multiple rounds.")
    while True:
        if not self.choose_difficulty():
            self.print_summary()
            return

        category = self.choose_category()
        if category is None:
            self.print_summary()
            return
        self.category = category

        if not self.play_round():
            self.print_summary()
            return

        if not self.ask_replay():
            self.print_summary()
            return
```